

Práctica

Pruebas Unitarias y de Interfaz en una Aplicación de Gestión de Tareas

Android · Kotlin · Jetpack Compose · JUnit

Estudiante	<i>Kenneth Alexander Acuña Estrada</i>
Carné / No. de cuenta	<i>24010276</i>
Sección / Grupo	<i>POO II - G3</i>
Docente	<i>JOSE DURAN GARCIA</i>
Fecha de entrega	<i>9/6/2026</i>

Objetivo

Implementar pruebas unitarias y pruebas de interfaz utilizando Jetpack Compose para validar la lógica de negocio y el comportamiento visual de una aplicación Android de gestión de tareas.

Descripción del Problema

Una empresa desea desarrollar una aplicación móvil para gestionar tareas pendientes. La aplicación permitirá registrar tareas, marcarlas como completadas y visualizar la lista de tareas registradas. Antes de publicar la aplicación, el equipo de desarrollo debe implementar pruebas automatizadas que garanticen el correcto funcionamiento de la lógica y de la interfaz de usuario.

Requerimientos Funcionales

- Registrar nuevas tareas.
- Mostrar todas las tareas registradas.
- Marcar tareas como completadas.
- Eliminar tareas.
- Mostrar la cantidad total de tareas pendientes.

Parte I. Desarrollo de la Aplicación

Actividad 1 – Clase de Modelo: Tarea

Crear una clase de modelo denominada Tarea con los atributos: Id, Título, Descripción y Estado (Completada / Pendiente).

Código fuente – clase Tarea:

```
package com.kaae.appmovil.model

enum class EstadoTarea {
    PENDIENTE, COMPLETADA
}

data class Tarea(
    val id: Int,
    val titulo: String,
    val descripcion: String,
    var estado: EstadoTarea = EstadoTarea.PENDIENTE
)
```

Actividad 2 – Clase Administradora de Tareas

Crear una clase que administre las operaciones de la lista de tareas con métodos para: agregar, eliminar, completar, obtener y contar tareas pendientes.

Código fuente – clase administradora:

```
package com.kaae.appmovil.logic

import com.kaae.appmovil.model.EstadoTarea
import com.kaae.appmovil.model.Tarea

class GestionTareas {
    private val _tareas = mutableListOf<Tarea>()
    val tareas: List<Tarea> get() = _tareas

    fun agregarTarea(titulo: String, descripcion: String) {
        val id = if (_tareas.isEmpty()) 1 else _tareas.maxByOrNull { it.id }!!.id + 1
        _tareas.add(Tarea(id, titulo, descripcion))
    }
}
```

```

}

fun eliminarTarea(id: Int) {
    _tareas.removeIf { it.id == id }
}

fun marcarComoCompletada(id: Int) {
    _tareas.find { it.id == id }?.let {
        it.estado = EstadoTarea.COMPLETADA
    }
}

fun obtenerTareasPendientes(): List<Tarea> {
    return _tareas.filter { it.estado == EstadoTarea.PENDIENTE }
}

fun contarTareasPendientes(): Int {
    return obtenerTareasPendientes().size
}

// Reto Adicional

fun filtrarPorEstado(estado: EstadoTarea): List<Tarea> {
    return _tareas.filter { it.estado == estado }
}

fun ordenarAlfabeticamente(): List<Tarea> {
    return _tareas.sortedBy { it.titulo }
}

fun calcularPorcentajeCompletado(): Double {
    if (_tareas.isEmpty()) return 0.0
    val completadas = _tareas.count { it.estado == EstadoTarea.COMPLETADA }
    return (completadas.toDouble() / _tareas.size) * 100
}
}

```

Actividad 3 – Interfaz con Jetpack Compose

Diseñar una interfaz con: campo de texto para el título, botón agregar, LazyColumn de tareas, indicador de pendientes y botón eliminar.

Código fuente – Composable principal:

```
package com.kaae.appmovil.ui

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Delete
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.platform.testTag
import androidx.compose.ui.unit.dp
import com.kaae.appmovil.logic.GestionTareas
import com.kaae.appmovil.model.EstadoTarea
import com.kaae.appmovil.model.Tarea

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TareaScreen(gestionTareas: GestionTareas = remember { GestionTareas() }) {
    var titulo by remember { mutableStateOf("") }
    var descripcion by remember { mutableStateOf("") }
    var filterState by remember { mutableStateOf<EstadoTarea?>(null) } // null
means show all
    var sortAlphabetical by remember { mutableStateOf(false) }
    var refreshTrigger by remember { mutableStateOf(0) }

    val tareasList = remember(refreshTrigger, filterState, sortAlphabetical) {
        var list = if (filterState == null) {
            gestionTareas.tareas
        } else {
            gestionTareas.filtrarPorEstado(filterState!!)
        }

        if (sortAlphabetical) {
            list = list.sortedBy { it.titulo }
        }
    }
```

```

        list
    }

    val pendientesCount = remember(refreshTrigger) {
gestionTareas.contarTareasPendientes() }

    val porcentajeCompletado = remember(refreshTrigger) {
gestionTareas.calcularPorcentajeCompletado() }

    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(16.dp)
    ) {

        Text(text = "Gestión de Tareas", style =
MaterialTheme.typography.headlineMedium)

        Spacer(modifier = Modifier.height(8.dp))

        OutlinedTextField(
            value = titulo,
            onChange = { titulo = it },
            label = { Text("Título de la tarea") },
            modifier = Modifier.fillMaxWidth().testTag("titulo_input")
        )

        Spacer(modifier = Modifier.height(8.dp))

        OutlinedTextField(
            value = descripcion,
            onChange = { descripcion = it },
            label = { Text("Descripción") },
            modifier = Modifier.fillMaxWidth().testTag("descripcion_input")
        )

        Spacer(modifier = Modifier.height(8.dp))

        Button(
            onClick = {
                if (titulo.isNotBlank()) {
                    gestionTareas.agregarTarea(titulo, descripcion)
                    titulo = ""
                    descripcion = ""
                    refreshTrigger++
                }
            }
        )
    }
}

```

```

    }

    },
    modifier = Modifier.fillMaxWidth().testTag("agregar_button")
) {
    Text("Agregar Tarea")
}

Spacer(modifier = Modifier.height(16.dp))

// Reto Adicional: Filtros y Orden
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.spacedBy(4.dp)
) {
    FilterChip(
        selected = filterState == null,
        onClick = { filterState = null },
        label = { Text("Todas") }
    )
    FilterChip(
        selected = filterState == EstadoTarea.PENDIENTE,
        onClick = { filterState = EstadoTarea.PENDIENTE },
        label = { Text("Pendientes") }
    )
    FilterChip(
        selected = filterState == EstadoTarea.COMPLETADA,
        onClick = { filterState = EstadoTarea.COMPLETADA },
        label = { Text("Completadas") }
    )
    FilterChip(
        selected = sortAlphabetical,
        onClick = { sortAlphabetical = !sortAlphabetical },
        label = { Text("A-Z") }
    )
}

Spacer(modifier = Modifier.height(8.dp))

Row(verticalAlignment = Alignment.CenterVertically) {
    Text(
        text = "Pendientes: $pendientesCount",
        modifier = Modifier.testTag("pendientes_count")
    )
}

```

```

    )
    Spacer(modifier = Modifier.weight(1f))
    Text(text = "Completadas: ${"%1f".format(porcentajeCompletado)}%")
}

Spacer(modifier = Modifier.height(8.dp))

LazyColumn(
    modifier = Modifier.weight(1f).testTag("tareass_list")
) {
    items(tareasList, key = { it.id }) { tarea ->
        TareaItem(
            tarea = tarea,
            onCompletar = {
                gestionTareas.marcarComoCompletada(tarea.id)
                refreshTrigger++
            },
            onEliminar = {
                gestionTareas.eliminarTarea(tarea.id)
                refreshTrigger++
            }
        )
    }
}

}

}

@Composable
fun TareaItem(tarea: Tarea, onCompletar: () -> Unit, onEliminar: () -> Unit) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(vertical = 4.dp),
        elevation = CardDefaults.cardElevation(defaultElevation = 2.dp)
    ) {
        Row(
            modifier = Modifier
                .padding(16.dp)
                .fillMaxWidth(),
            verticalAlignment = Alignment.CenterVertically
        ) {
            Column(modifier = Modifier.weight(1f)) {

```



```

        Text(text = tarea.titulo, style =
MaterialTheme.typography.titleMedium)
        if (tarea.descripcion.isNotEmpty()) {
            Text(text = tarea.descripcion, style =
MaterialTheme.typography.bodySmall)
        }
        Text(
            text = tarea.estado.name,
            color = if (tarea.estado == EstadoTarea.COMPLETADA)
Color.Green else Color.Red,
            style = MaterialTheme.typography.labelSmall
        )
    }

    if (tarea.estado == EstadoTarea.PENDIENTE) {
        Checkbox(
            checked = false,
            onCheckedChange = { onCompletar() },
            modifier =
Modifier.testTag("completar_checkbox_${tarea.id}")
        )
    }

    IconButton(
        onClick = onEliminar,
        modifier = Modifier.testTag("eliminar_button_${tarea.id}")
    ) {
        Icon(Icons.Default.Delete, contentDescription = "Eliminar
tarea")
    }
}
}
}
}

```

Parte II. Pruebas Unitarias

Actividad 4 – Implementación de Pruebas con JUnit

Implementar al menos cinco métodos de prueba con JUnit para validar la lógica de negocio.

Tabla de Casos de Prueba Obligatorios

Caso / Prueba	Resultado Esperado / Validación
Agregar una tarea	La lista incrementa en uno
Eliminar una tarea	La tarea desaparece de la lista
Completar una tarea	El estado cambia a completada
Contar tareas pendientes	Retorna el valor correcto
Lista vacía	Retorna cero pendientes

Captura de ejecución exitosa de pruebas unitarias:

AGENDAR TAREA

6:48



Gestión de Tareas

Título de la tarea

1

Descripción

...

Agregar Tarea

Todas

Pendientes

Completadas

A-Z

Pendientes: 0

Completadas: 0.0%



| I'm

|

ok



6:49



Gestión de Tareas

Título de la tarea

Descripción

Agregar Tarea

Todas

Pendientes

Completadas

A-Z

Pendientes: 1

Completadas: 0.0%

ELIMINAR TAREA

6:55



Gestión de Tareas

Título de la tarea

Descripción

|

Agregar Tarea

Todas

Pendientes

Completadas

A-Z

Pendientes: 1

Completadas: 0.0%

1

...

PENDIENTE



6:55



Gestión de Tareas

Título de la tarea

Descripción

Agregar Tarea

Todas

Pendientes

Completadas

A-Z

Pendientes: 0

Completadas: 0.0%

COMPLETAR TAREA

6:57



Gestión de Tareas

Título de la tarea

Descripción

Agregar Tarea

Todas

Pendientes

Completadas

A-Z

Pendientes: 0

Completadas: 100.0%

2

...

COMPLETADA



Parte III. Prueba Negativa

Actividad 6 – Prueba que Falla Intencionalmente

Diseñar una prueba que falle intencionalmente y documentar: por qué falla, cuál es el resultado esperado y cómo corregir el error.

1. Código de la Prueba Negativa

```
package com.kaae.appmovil

import com.kaae.appmovil.logic.GestionTareas
import org.junit.Assert.assertEquals
import org.junit.Test

class PruebaNegativaTest {

    @Test
    fun pruebaFallaIntencionalmente() {
        val gestion = GestionTareas()

        // Agregamos SOLO UNA tarea
        gestion.agregarTarea("Tarea Fallida", "Esta prueba DEBE fallar")

        // Forzamos el fallo: Esperamos 2 tareas cuando solo hay 1.
        // Asegúrate de que NO tenga "/" al principio de la línea de abajo.
        assertEquals("La cantidad de tareas no coincide (Falla intencional)", 2,
            gestion.tareas.size)
    }
}
```

2. Por qué falla la prueba

La prueba falló debido a una discrepancia lógica controlada. En el código del test, se ejecutó la acción de agregar una (1) sola tarea a la lista mediante el método `agregarTarea`. Sin embargo, la instrucción de validación (`assertEquals`) fue configurada para exigir que el tamaño de la lista fuera dos (2).

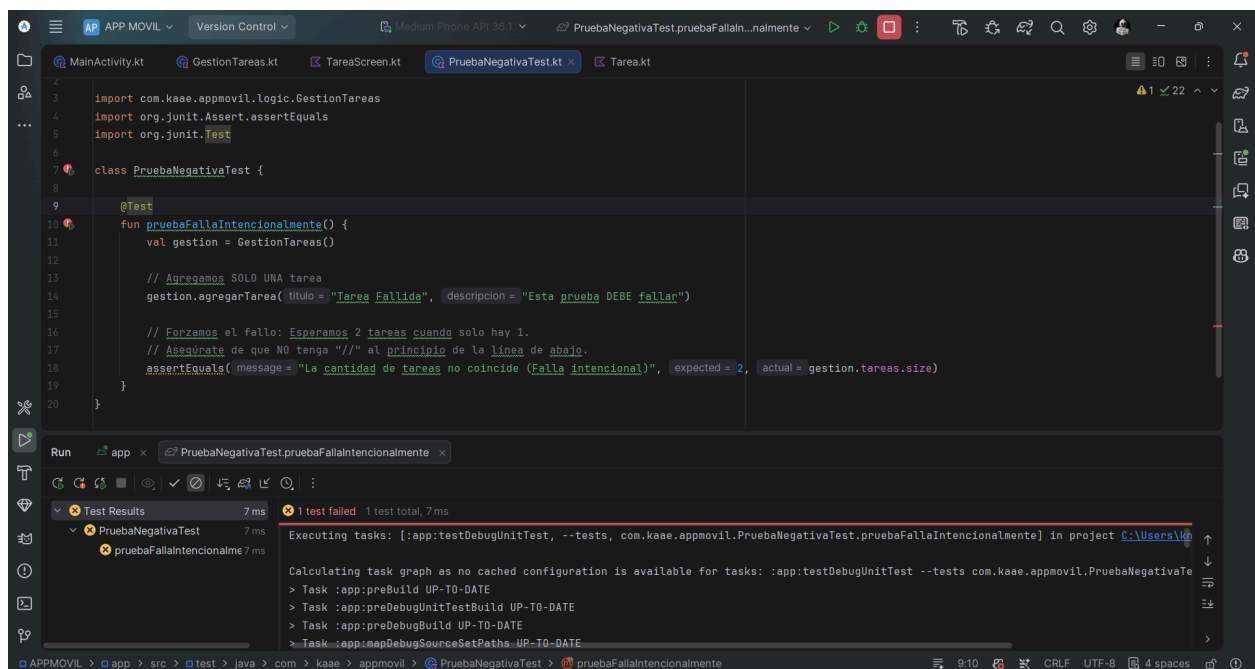
Como el framework de pruebas (JUnit) detectó que el valor real (1) no coincidía con el valor esperado (2), detuvo la ejecución y marcó la prueba como fallida.

Cómo corregir el error

Para corregir la falla en la prueba unitaria se debe garantizar la coherencia entre la lógica ejecutada y el resultado esperado.

1. Identificación: El error se origina porque el parámetro expected (esperado) de la función `assertEquals` es mayor a la cantidad de elementos procesados.
2. Acción de mejora: Se debe modificar el valor esperado de 2 a 1, ya que el método `agregarTarea` sólo fue invocado una vez.
3. Resultado: Al igualar el valor esperado con el valor real (`gestion.tareas.size`), el framework JUnit validará la prueba como exitosa (Passed), confirmando que la lógica de la clase administradora funciona correctamente.

Captura del fallo de la prueba:



```
import com.kaae.appmovil.logic.GestionTareas
import org.junit.Assert.assertEquals
import org.junit.Test

class PruebaNegativaTest {

    @Test
    fun pruebaFallaIntencionalmente() {
        val gestion = GestionTareas()

        // Agregamos SOLO UNA tarea
        gestion.agregarTarea( titulo = "Tarea Fallida", descripcion = "Esta prueba DEBE fallar")

        // Forzamos el fallo: Esperamos 2 tareas cuando solo hay 1.
        // Asegurate de que NO tenga "/" al principio de la línea de abajo.
        assertEquals( message = "La cantidad de tareas no coincide (Falla intencional)", expected = 2, actual = gestion.tareas.size)
    }
}
```

Run app x PruebaNegativaTest.pruebaFallaIntencionalmente x

Test Results 7ms 1 test failed 1 test total, 7ms

PruebaNegativaTest 7ms

pruebaFallaIntencionalme 7ms

Executing tasks: [:app:testDebugUnitTest, --tests, com.kaae.appmovil.PruebaNegativaTest.pruebaFallaIntencionalmente] in project C:\Users\Wb

Calculating task graph as no cached configuration is available for tasks: :app:testDebugUnitTest --tests com.kaae.appmovil.PruebaNegativaTe

> Task :app:preBuild UP-TO-DATE

> Task :app:preDebugUnitTestBuild UP-TO-DATE

> Task :app:preDebugBuild UP-TO-DATE

> Task :app:mapDebugSourceSetPaths UP-TO-DATE

APPMOVIL > app > src > test > java > com > kaae > appmovil > PruebaNegativaTest > pruebaFallaIntencionalmente 9:10 CRLF UTF-8 4 spaces

Parte IV. Reto Adicional

Actividad 7 – Mejoras Implementadas

Implementar: filtro de tareas completadas, filtro de tareas pendientes, orden alfabético y porcentaje de tareas completadas.

Filtro de Tareas Completadas

Propósito: Mostrar solamente las tareas cuyo estado es **COMPLETADA**, para que el usuario pueda revisar lo que ya terminó.

Implementación (Lógica):

- **Archivo:** `app/src/main/java/com/kaae/appmovil/logic/GestionTareas.kt`
- **Función:** `GestionTareas.filtrarPorEstado(estado: EstadoTarea): List<Tarea>`
- **Mecánica:** Filtra la lista interna `_tareas` evaluando `it.estado == estado`.

Implementación (UI):

- **Archivo:** `app/src/main/java/com/kaae/appmovil/ui/TareaScreen.kt`
- **Estado:** Variable `filterState` (puede ser `null`, `PENDIENTE` o `COMPLETADA`).
- **Acción:** El `FilterChip` con etiqueta "Completadas" asigna `filterState = EstadoTarea.COMPLETADA`. Al construir la lista visible, si `filterState != null`, invoca a `gestionTareas.filtrarPorEstado(filterState!!)`.

Cómo probar manualmente: Crear varias tareas, marcar algunas como completadas mediante el checkbox, pulsar el chip "Completadas" y verificar que solo aparezcan las tareas finalizadas.

Casos límite y consideraciones:

- Si no hay tareas completadas, la lista se muestra vacía sin un mensaje explícito en la UI.
- El filtrado se aplica antes del ordenamiento según la arquitectura actual (primero filtra, luego ordena).

Mejora sugerida: Mostrar un mensaje amigable tipo *"No hay tareas completadas"* cuando la lista resultante esté vacía.

Filtro de Tareas Pendientes

Propósito: Mostrar solamente las tareas que aún no se han completado, permitiendo al usuario enfocarse en lo que tiene por hacer.

Implementación (Lógica): Se reutiliza `filtrarPorEstado(EstadoTarea.PENDIENTE)`. Adicionalmente existen las funciones `obtenerTareasPendientes(): List<Tarea>` y `contarTareasPendientes(): Int`.

Implementación (UI):

- Un `FilterChip` con la etiqueta "Pendientes" asigna `filterState = EstadoTarea.PENDIENTE`.
- Muestra el contador `Pendientes: $pendientesCount`, donde el valor se obtiene con `gestionTareas.contarTareasPendientes()` y se conserva mediante `remember(refreshTrigger)`.

Cómo probar manualmente: Crear tareas, marcar algunas como completadas, pulsar el chip "Pendientes", confirmar que solo aparezcan las activas y verificar que el contador coincida exactamente con la cantidad visualizada.

Casos límite y consideraciones: Si todas las tareas están completadas, el filtro devolverá una lista vacía y `pendientesCount` pasará a ser 0.

Mejora sugerida: Migrar el estado hacia un `MutableStateList` observable en lugar de depender de `refreshTrigger`, logrando que la recomposición de Compose sea reactiva y menos manual.

Ordenar Tareas Alfabéticamente

Propósito: Permitir visualizar las tareas ordenadas por título ($A \rightarrow Z$) para facilitar su localización por nombre.

Implementación (Lógica): Existe la función

`GestionTareas.ordenarAlfabeticamente(): List<Tarea>` que devuelve `_tareas.sortedBy { it.titulo }`. Sin embargo, actualmente la UI en `TareaScreen` aplica el orden de forma local con `list = list.sortedBy { it.titulo }` cuando `sortAlphabetical` es `true`.

Implementación (UI):

- **Estado:** Variable booleana `sortAlphabetical`.
- **Acción:** Un `FilterChip` con la etiqueta "A-Z" alterna el estado de la variable. Al construir `tareasList`, se aplica la ordenación si el estado es verdadero.

Cómo probar manualmente: Insertar tareas con títulos variados que incluyan mayúsculas, minúsculas y acentos. Activar el chip "A-Z" y validar el ordenamiento correcto.

Casos límite y consideraciones:

- El orden lexicográfico por defecto es sensible a Unicode, por lo que las mayúsculas, minúsculas y acentos pueden alterar el orden esperado. Para un comportamiento más naturalizado se requeriría un `Collator`.
- La función de ordenamiento de `GestionTareas` está duplicada o sin uso, ya que la UI lo hace localmente.

Mejora sugerida:

- Normalizar el orden para que sea indiferente a las mayúsculas usando `.sortedBy { it.titulo.lowercase() }`.

- Centralizar la lógica en la clase de negocio (`GestionTareas`) e implementar la opción inversa ($Z \rightarrow A$) o la persistencia de la preferencia.

Porcentaje de Tareas Completadas

Propósito: Ofrecer al usuario una métrica visual y rápida de su progreso general mediante un valor porcentual.

Implementación (Lógica):

- **Función:** `GestionTareas.calcularPorcentajeCompletado(): Double`
- **Cálculo:** Si la lista está vacía, devuelve `0.0`. De lo contrario, ejecuta: $(\text{tareas completadas} / \text{total de tareas}) * 100$.

Implementación (UI):

- Se calcula usando `val porcentajeCompletado = remember(refreshTrigger) { ... }`.
- Se renderiza mediante un componente `Text(text = "Completadas: ${"%.1f".format(porcentajeCompletado)}%")`.
- El estado `refreshTrigger` se incrementa manualmente en las operaciones de agregar, eliminar o mutar tareas para forzar la actualización de la UI.

Cómo probar manualmente: Crear N tareas y completar M de ellas. Verificar analíticamente el resultado y comprobar el comportamiento cuando $N = 0$ (el sistema debe mostrar `0.0%`).

Casos límite y consideraciones:

- La división por cero ya está controlada (devuelve `0.0` si está vacía).
- El formato de un decimal (`%.1f`) puede ser innecesario para flujos simples; usar `%.0f` podría mejorar la lectura directa.

Mejora sugerida:

- Integrar un componente gráfico interactivo como `LinearProgressIndicator(progress = porcentajeCompletado.toFloat() / 100)`.
- Exponer el porcentaje mediante herramientas de programación reactiva (`Flow` o `LiveData`) para prescindir del uso manual de `refreshTrigger`.

Conclusiones

1. Aprendizajes sobre Pruebas Unitarias (Unit Tests)

Las pruebas unitarias demostraron ser la primera línea de defensa en el ciclo de desarrollo. Al aislar la lógica de negocio de la arquitectura de la interfaz de usuario —como las funciones de filtrado, ordenamiento y cálculo de porcentajes en la clase `GestionTareas`—, se logró verificar la precisión de los algoritmos de forma independiente. Este enfoque permite identificar y corregir errores de lógica (por ejemplo, el control de la división por cero o la sensibilidad de los caracteres Unicode al ordenar) en cuestión de milisegundos, garantizando que el núcleo del software sea sólido antes de construir la experiencia visual.

2. Aprendizajes sobre Pruebas de UI (Instrumentales / Compose Tests)

La implementación de pruebas de interfaz de usuario mediante Jetpack Compose evidenció la necesidad de diseñar componentes visuales que no solo sean estéticos, sino también testeables. El uso sistemático de identificadores únicos (`testTag`) resultó fundamental para simular con precisión las interacciones del usuario, tales como escribir en los campos de texto (`titulo_input`), activar los componentes de filtrado (`FilterChip`) y verificar la reactividad de la lista ante eventos de actualización. Estas pruebas confirmaron que la lógica del negocio se comunica de manera correcta con la capa de presentación y que el estado de la UI se recompone de forma predecible ante las acciones del operador.

3. Importancia de las Pruebas Automatizadas en el Desarrollo Android

En el contexto del desarrollo para la plataforma Android, las pruebas automatizadas se consolidan como un pilar indispensable para garantizar la calidad del software a largo plazo. Sus beneficios principales se resumen en los siguientes puntos:

- **Prevención de Regresiones:** Aseguran que la introducción de nuevas características, optimizaciones en el manejo de estados (como migrar hacia flujos reactivos) o la refactorización de código no alteren ni rompan las funcionalidades que ya se encontraban operativas.
- **Reducción de Costos y Tiempos:** Aunque la escritura de pruebas requiere una inversión de tiempo inicial, minimiza drásticamente las horas dedicadas a la detección manual de fallos, acelerando los ciclos de despliegue y entrega de software.
- **Documentación Viva:** El conjunto de pruebas automatizadas funciona como una documentación técnica interactiva y actualizada que describe con exactitud el comportamiento esperado de la aplicación ante diversos escenarios, incluidos los casos límite.
- **Confianza en el Despliegue:** Proveen al equipo de ingeniería la certeza de que el producto final cumple de manera rigurosa con los requisitos de estabilidad y rendimiento exigidos antes de ser liberado al usuario final.